C++指针：经典考点解析

我来把这两个问题一次性讲清楚，非常关键，也是C++指针里的**经典考点**。

1. 为什么输出 `char*` 要加 `(void*)`？

一句话：

因为 `cout` 对 `char*` 有特殊待遇，它会把 `char*` 当成「字符串」，而不是「地址」。

举个例子

Code block

```
char ch = 'a';
char* p = &ch;

cout << p;    // 不会输出地址!
```

发生了什么？

- `cout` 看到 `char*`，会认为你要输出一个**C风格字符串**
- 于是它从 `p` 指向的地址开始，**一直往后读字节，直到遇到 `\0` 才停**
- 但你这里只有一个 `'a'`，后面全是乱码内存
- 结果就是：输出一堆乱码，而不是地址

怎么让它输出地址？

强制转换成 `void*`：

Code block

```
cout << (void*)p;
```

- `void*` 是**通用指针类型**
- `cout` 对 `void*` 没有特殊处理，只会老老实实输出**内存地址**

总结

- `int*` / `float*` / `double*` → 直接输出地址
- `char*` → 默认输出字符串
- 想输出 `char*` 的地址 → 必须强转 `(void*)`

2. 为什么 `p + 3` 合法，`p + 4` 不合法?

先看你的代码：

Code block

```
int arr[3] = {1,2,3};
int* p = arr;
```

数组下标：`arr[0]` `arr[1]` `arr[2]`

一共 3 个元素。

C++ 标准有一条明确规则：

指针允许指向「数组最后一个元素的下一个位置」，这个位置叫「尾后指针」，是合法的。

也就是说：

- 合法范围：`p + 0` ～ `p + 3`
- `p+3` 就是**尾后指针**

为什么它合法?

主要用来做**边界判断**，比如遍历数组：

Code block

```
while(p < p + 3) { ... }
```

标准允许你**生成、比较**这个尾后指针，只是**不允许解引用** `*(p+3)`。

那 `p + 4` 为什么非法?

因为它**越过了尾后指针**，进入了完全不属于数组的内存区域。

这属于**未定义行为**，程序随时可能崩溃。

最终极简总结

1. `(void*)` **是为了让** `char*` **输出地址，而不是输出字符串**
2. `p+3` **合法**：它是数组尾后指针，C++专门允许用来判断边界
3. `p+4` **非法**：超出尾后，访问未知内存，属于越界

（注：文档部分内容可能由 AI 生成）